EXPERIMENT – 9

ROLL NO: 240701359 **NAME:** Nikhil Vinayak P

USER INTERFACE AND DESIGN

Register Form Validation

AIM:

To design a registration form that validates user input such as name, email, phone number, and password using HTML, CSS, JavaScript, and Validator.js, and displays appropriate error and success messages.

EXPLANATION

A registration form is used to collect user details in web applications. However, users may enter invalid or incomplete data. To ensure correctness, form validation is performed before submission.

In this experiment:

- HTML is used to create the structure of the registration form.
- CSS is used to design a clean and user-friendly interface.
- JavaScript is used to validate user inputs.
- Validator.js is used to simplify validation of common inputs such as email and empty fields.

The form checks the following conditions:

- Name → Must contain only alphabets and spaces
- Email → Must follow valid email format
- Phone Number → Must be 10 digits and start with 6–9
- Password → Must contain at least 6 characters

Error messages are displayed below each field, and the form is prevented from submitting until all inputs are valid.

PROCEDURE

1. Create an HTML form with input fields for name, email, phone number, and password.
2. Style the form using CSS for a clean and modern appearance.
3. Include the Validator.js library using CDN.
4. Write JavaScript to validate each input field.
5. Display error messages below inputs when validation fails.
6. Remove error messages when the user corrects the input.
7. Prevent form submission if any field is invalid.
8. Display a success message below the button when all inputs are valid.

HTML (page.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Register</title>
  <link rel="stylesheet" href="style.css">

  <!-- Validator.js -->
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/validator/13.9.0/validator.min.js"></script>
</head>
<body>

<div class="container">
  <h2>Create Account</h2>

  <form id="form" novalidate>

    <div class="form-group">
      <input type="text" id="name" placeholder="Full Name">
      <span class="error" id="nameError"></span>
    </div>

    <div class="form-group">
      <input type="text" id="email" placeholder="Email">
      <span class="error" id="emailError"></span>
    </div>

    <div class="form-group">
      <input type="text" id="phone" placeholder="Phone Number">
      <span class="error" id="phoneError"></span>
    </div>

    <div class="form-group">
      <input type="password" id="password" placeholder="Password">
      <span class="error" id="passwordError"></span>
    </div>

    <button type="submit">Register</button>

    <!-- Success message -->
    <p id="successMsg" class="success"></p>

  </form>
</div>
```

```
<script src="script.js"></script>

</body>
</html>
```

CSS (style.css)

```
body {
   font-family: Arial, sans-serif;
   background: #f3f3f3;
   display: flex;
   justify-content: center;
   align-items: center;
   height: 100vh;
}

/* Card */
.container {
   background: #fff;
   padding: 30px;
   width: 300px;
   border-radius: 12px;
   box-shadow: 0 6px 20px rgba(0,0,0,0.08);
}

h2 {
   text-align: center;
   margin-bottom: 20px;
   color: #222;
}

.form-group {
   margin-bottom: 15px;
}

/* Inputs */
input {
   width: 100%;
   padding: 10px;
   border: 1px solid #ccc;
   border-radius: 6px;
   outline: none;
}

input:focus {
   border-color: #333;
```

```
}

/* Error message */
.error {
  color: #e74c3c;
  font-size: 12px;
  display: block;
  margin-top: 4px;
}

/* Error style */
input.error-border {
  border: 1px solid #e74c3c;
  background: #fff5f5;
}

/* Success message */
.success {
  color: #2ecc71;
  font-size: 13px;
  text-align: center;
  margin-top: 10px;
}

/* Button */
button {
  width: 100%;
  padding: 10px;
  background: #222;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
}

button:hover {
  background: #444;
}
```

JavaScript (script.js)

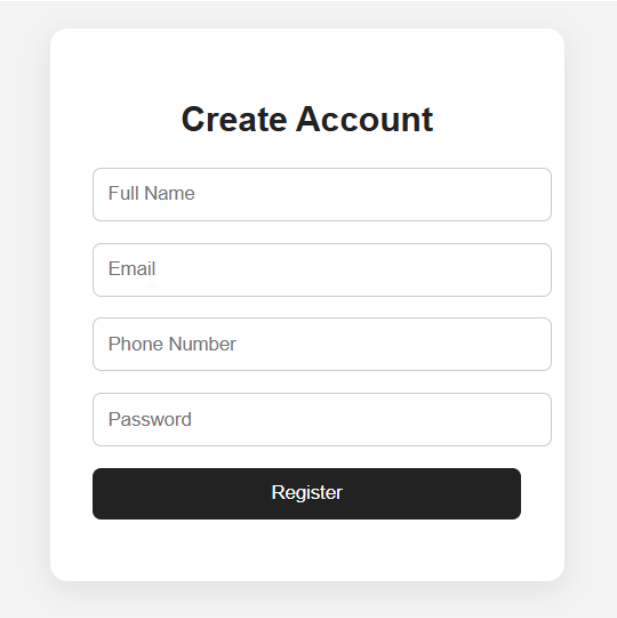
```
const form = document.getElementById("form");

const name = document.getElementById("name");
const email = document.getElementById("email");
const phone = document.getElementById("phone");
const password = document.getElementById("password");
```

```
const successMsg = document.getElementById("successMsg");

/* 🔷 Live validation */
[name, email, phone, password].forEach(input => {
  input.addEventListener("input", () => {
    validateField(input);
  });
});

/* 🔷 Submit validation */
form.addEventListener("submit", function(e) {

  let isValid = true;
  successMsg.textContent = "";

  [name, email, phone, password].forEach(input => {
    if (!validateField(input)) {
      isValid = false;
    }
  });

  if (!isValid) {
    e.preventDefault();
  } else {
    e.preventDefault(); // stop refresh
    successMsg.textContent = "Registration successful!";
    form.reset();
  }
});

/* 🔷 Validation logic */
function validateField(input) {

  let value = input.value.trim();
  clearError(input);

  if (input.id === "name") {
    if (validator.isEmpty(value)) {
      setError(input, "nameError", "Name is required");
      return false;
    }
    if (!validator.isAlpha(value.replace(/\s/g, ''))) {
      setError(input, "nameError", "Only alphabets allowed");
      return false;
    }
  }
```

```
    if (input.id === "email") {
      if (validator.isEmpty(value)) {
        setError(input, "emailError", "Email is required");
        return false;
      }
      if (!validator.isEmail(value)) {
        setError(input, "emailError", "Invalid email");
        return false;
      }
    }

    if (input.id === "phone") {
      if (validator.isEmpty(value)) {
        setError(input, "phoneError", "Phone is required");
        return false;
      }
      if (!/^[6-9]\d{9}$/.test(value)) {
        setError(input, "phoneError", "Invalid phone number");
        return false;
      }
    }

    if (input.id === "password") {
      if (validator.isEmpty(value)) {
        setError(input, "passwordError", "Password is required");
        return false;
      }
      if (!validator.isLength(value, { min: 6 })) {
        setError(input, "passwordError", "Min 6 characters");
        return false;
      }
    }

    return true;
}

/* ◈ Show error */
function setError(input, id, message) {
    document.getElementById(id).textContent = message;
    input.classList.add("error-border");
}

/* ◈ Clear error */
function clearError(input) {
    const errorId = input.id + "Error";
    document.getElementById(errorId).textContent = "";
    input.classList.remove("error-border");}
```

SCREENS WITH EXPLANATION

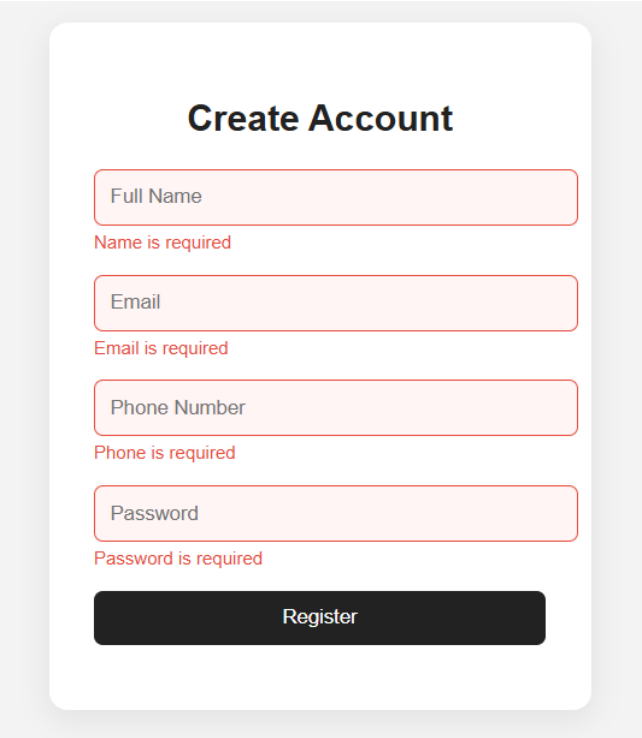
1. Register Form Screen



This is the main interface of the application. It contains input fields for Name, Email, Phone Number, and Password, along with a Register button.

Each field is clearly visible and aligned in a centered layout. Initially, no error messages are displayed.

2. Empty Submission (All Fields Empty)



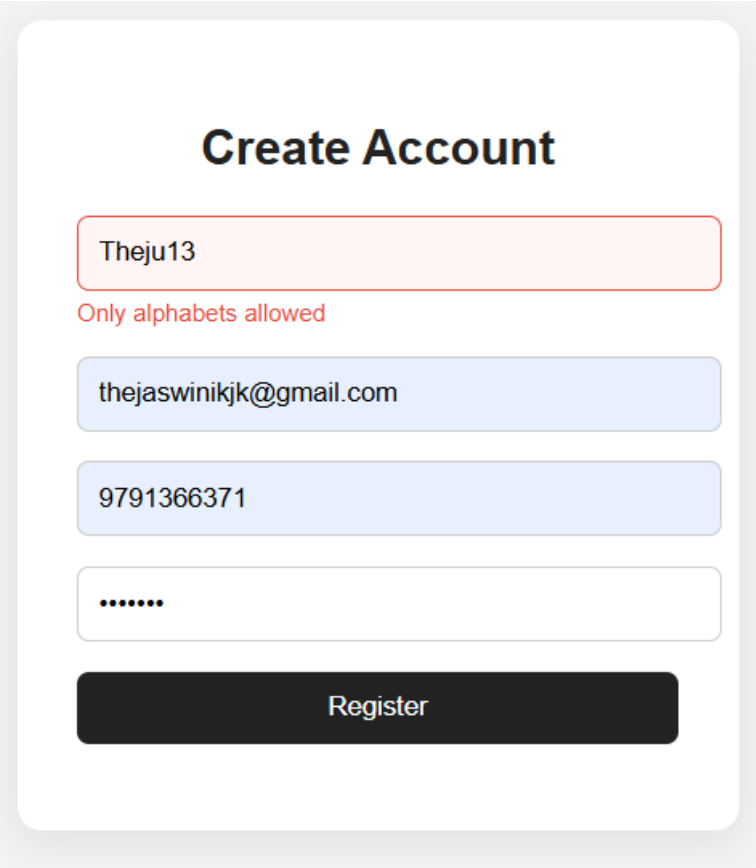
This screen appears when the user clicks the Register button without entering any data.

The system displays error messages for all fields:

- Name → “Name is required”
- Email → “Email is required”
- Phone → “Phone is required”
- Password → “Password is required”

The input fields are highlighted to indicate invalid entries, and the form is not submitted.

3. Name Validation Error Screen



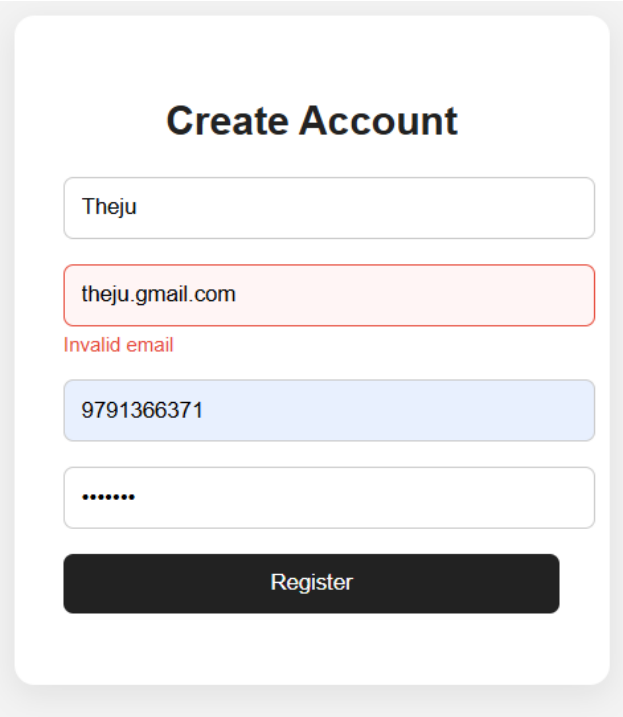
This screen appears when the user enters an invalid name such as “John123” or “Juju_01”.

The validation logic checks that the name should contain only alphabets and spaces.
If invalid characters are detected, an error message is displayed:

- “Only alphabets allowed”

The input field is highlighted to show the error.

4. Email Validation Error Screen

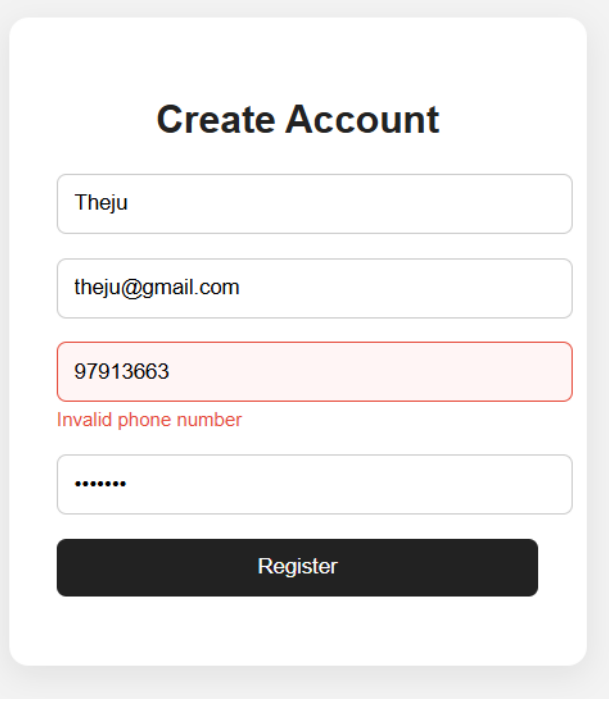


This screen appears when the user enters an invalid email such as “abcgmail.com”.

The validation checks whether the email contains proper format including “@” and domain. If invalid, the following message is displayed:

- “Invalid email”

5. Phone Number Validation Error Screen



This screen appears when the user enters an incorrect phone number such as “12345”.

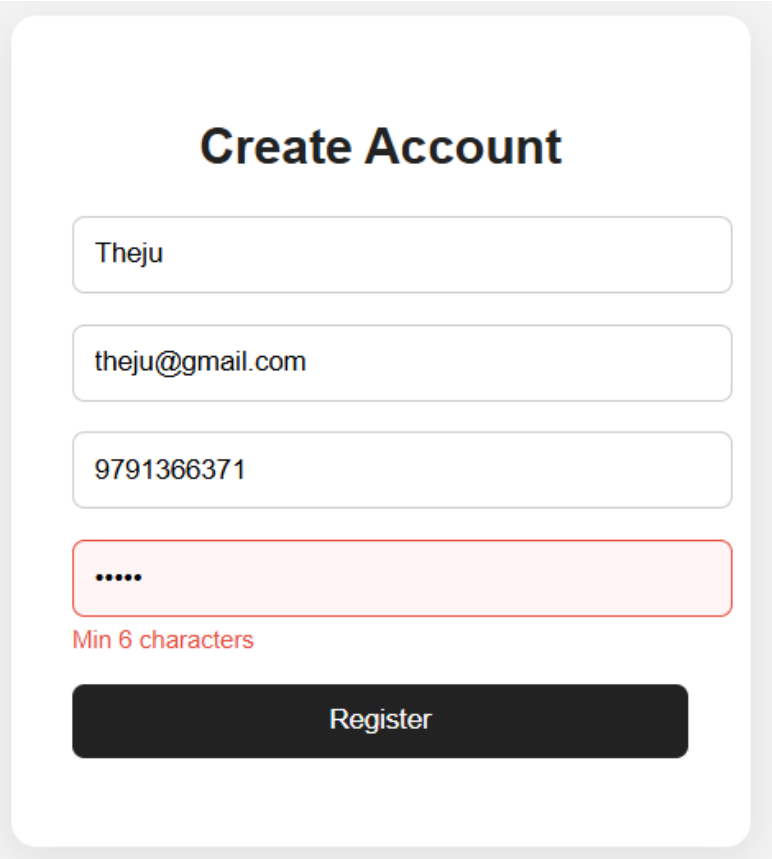
The validation ensures that:

- The number has exactly 10 digits
- It starts with digits 6–9

If not satisfied, the error message displayed is:

- “Invalid phone number”

6. Password Validation Error Screen



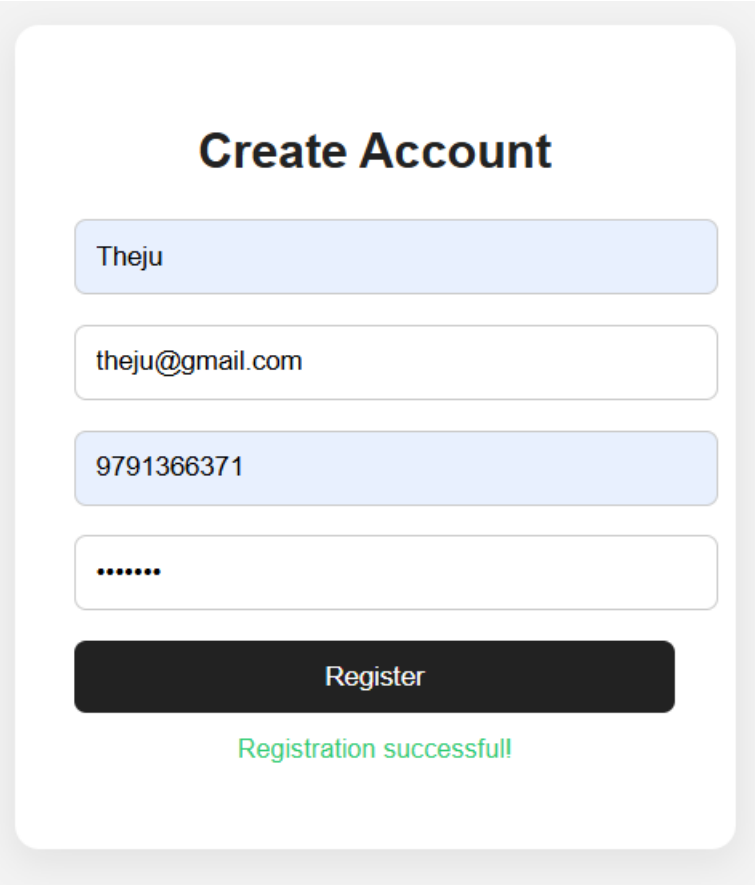
This screen appears when the user enters a weak or short password such as “123”.

The validation checks that the password must contain at least 6 characters.

Error displayed:

- “Min 6 characters”

7. Successful Registration Screen



This screen appears when all inputs are valid.

No error messages are displayed, and the form shows a success message below the Register button:

- “Registration successful!”

The form is cleared after successful submission, indicating that all validation conditions are satisfied.